

bpplay



A bit-perfect, minimalist music player for macOS

User Manual

V0.9.7

Core edition · 2026

Written by:

Ferenc Koscsó

Koscso Media Engineering Kft.

2026.

Contents

Contents.....	2
1. Introduction.....	4
1.1 Philosophy.....	4
2. Installation.....	5
2.1 Opening the disk image.....	5
2.2 Verifying the download.....	5
2.3 First launch (Gatekeeper).....	5
2.4 Building from source (optional).....	6
3. Quick start.....	7
3.1 Listing the devices.....	7
3.2 Playing a single file.....	7
3.3 Playing several files as a playlist (gapless).....	7
3.4 Playing a whole folder.....	7
3.5 Minimum system requirements.....	8
3.5.1 Operating system.....	8
3.5.2 Processor.....	8
3.5.3 Memory (RAM).....	8
3.5.4 Audio hardware (DAC).....	8
3.5.5 Storage.....	9
4. Command line options.....	10
4.1 Exit codes.....	10
5. Transport controls during playback.....	11
5.1 The status line.....	11
5.2 Click-free stopping and pausing.....	11
6. Supported formats.....	12
6.1 DSD and DoP.....	12
The DoP rates.....	12
6.2 Known limitation — macOS USB and 32-bit integer.....	13
7. The whole-track-in-RAM model.....	14
7.1 Why is this good?.....	14
7.2 About memory use.....	14
8. Mixed-format playlists.....	15
9. Drag-and-drop launching.....	16
9.1 Building the Automator application.....	16
9.2 Playback by right-clicking (Quick Action).....	16
10. Troubleshooting.....	18
11. Feature summary and validated state.....	19
11.1 Supported formats.....	19
11.2 The signal path.....	19
11.3 The whole-track-in-RAM model.....	19

11.4 DSD handling.....	19
11.5 Playback and convenience.....	19
11.6 Engineering character.....	20
11.7 Validated state.....	20
12. About bppplay — philosophy and architecture.....	21
12.1 The starting point: analogue minimalism.....	21
12.2 What a player usually does — and what bppplay does not.....	22
12.3 The bit-perfect signal path.....	22
12.4 The electrical benefit of the RAM model.....	22
12.5 DoP and DSD.....	23
12.6 Format handling and gapless playback.....	23
12.7 What is deliberately left out.....	23
12.8 The computer as a noise source — and the clock question.....	23
12.9 A reference, not a competitor.....	24
13. Licence terms and copyright notice.....	25

1. Introduction

bpplay is a minimalist, bit-perfect music player for macOS. It has a single purpose: to deliver the samples of a music file to the DAC untouched, over the shortest possible deterministic path, while generating as little internal operating noise as possible. There is no software volume, no dither, no resampling and no unnecessary conversion in the signal path. What the production captured is what reaches the DAC — bit for bit.

This manual covers **version 0.9.x of the Core edition**: the puritan variant that plays local files, whose greatest value is its simplicity and transparency.

The project is open source under the GPL-3.0 licence: <https://github.com/ferenckoscso/bpplay>

1.1 Philosophy

bpplay follows the principle of the shortest deterministic signal path. Ordinary macOS players work through the CoreAudio system mixer, which may resample, and which introduces floating-point mixing and software volume into the chain. **bpplay** bypasses all of it.

Format matching happens in a single lossless step, at load time, off the realtime thread — so during playback the system does as little work as possible.

2. Installation

bpplay arrives as a disk image (.dmg). Installation is a single drag — there is no installer wizard and no background service.

The official release can be downloaded from the GitHub Releases page:

<https://github.com/ferencscso/bpplay/releases/tag/v0.9>

Upgrading from a previous version: the disk image now includes a "Remove old version.command" helper — double-click it to find and (after confirming) move previous bpplay copies to the Trash before installing the new one. It is optional, just a convenience.

2.1 Opening the disk image

1. Double-click the downloaded .dmg file. A window opens with its contents.
2. Drag the „bpplay” file anywhere you like — for example onto the Desktop, or into a folder of your own. (If you want to run it from anywhere in Terminal, /usr/local/bin is a good place.)
3. The disk image also contains a "bpplay drop" app — drag this to the same place too (e.g. your Desktop). It lets you start playback without Terminal commands, just by dropping files/folders onto it (see Chapter 9).
4. The disk image also contains the licence and a first-launch guide.

2.2 Verifying the download

The Releases page carries the matching SHA256 checksum next to the .dmg. It is worth verifying:

```
shasum -a 256 bpplay-0.9.7-macos.dmg
```

The value you get must match the contents of the published .sha256 file.

2.3 First launch (Gatekeeper)

bpplay and bpplay drop have been Apple Developer ID signed and notarised since v0.9.7 — no "unidentified developer" warning appears for either; they simply launch.

bpplay play.workflow (the right-click Quick Action) is signed, but Apple's notarisation tooling specifically doesn't support Quick Action (.workflow) files, so it will still warn on first use. This is safe, it just can't technically go through Apple's notarisation process. Work around it once:

- From Terminal (recommended):

```
xattr -dr com.apple.quarantine ~/Library/Services/"bpplay play.workflow"
```

- From System Settings: try opening it from the right-click menu → "cannot be opened" error → dismiss ("Done") → System Settings → Privacy & Security → scroll down → "Open Anyway" button next to the item.

(The older "right-click → Open → Open" trick no longer works since macOS Sequoia — the warning dialog no longer has a button to allow it.)

2.4 Building from source (optional)

The philosophy of **bpplay** is transparency, so the full source code is publicly available. If you want to build it yourself — to verify it, or to customise it — the source builds with zero external dependencies, using the Xcode command line tools:

```
git clone https://github.com/ferenckoscs/bpplay.git
cd bpplay/src
make
```

or by hand:

```
clang -O2 bpplay.c -o bpplay \
-framework CoreAudio -framework CoreFoundation -framework IOKit
```

For a universal (Apple Silicon + Intel) binary:

```
make universal
```

Most users will not need this — the ready-made **bpplay** in the disk image is usable straight away.

3. Quick start

bpplay starts from Terminal.

Open Terminal (Spotlight → „Terminal“, or the Applications/Utilities folder) and change into the folder where you put bpplay. If it is on the Desktop, for example:

```
cd ~/Desktop
```

From there the commands use the `./bpplay` form (the `./` means „from the current folder“).

Tip — drag & drop into Terminal: a single file, several files or a whole folder can be dragged from Finder straight into the Terminal window. macOS then types the full path for you (separated by spaces for multiple items). That way you never have to type long paths by hand: type the beginning of the command (e.g. `./bpplay -d 1`), then drag the files in and the line completes itself.

3.1 Listing the devices

First, look at which output devices are available and what their indices are:

```
./bpplay -l
```

Note the index of your DAC from the list (e.g. 1).

3.2 Playing a single file

```
./bpplay -d 1 music.wav
```

The `-d 1` selects the DAC by index. If you leave it out, playback goes to the default output.

3.3 Playing several files as a playlist (gapless)

```
./bpplay -d 1 01.flac 02.flac 03.flac
```

Files of the same format play back gapless. Where the format differs, **bpplay** performs a clean switch (see chapter 8).

A single invocation plays at most **256 files**; anything beyond that is skipped, and the program says so.

3.4 Playing a whole folder

```
./bpplay -d 1 -dir /Users/you/Music/Album
```

The playable files of the folder (`.wav`, `.flac`, `.aif`, `.aiff`, `.dsf`) are played sorted by name. The walk is **recursive and album-ordered**: first the files of that folder by name, then the subfolders by name, descending into each. So dropping a collection folder plays every album in its own track order, one after another.

The `-dir` option **appends** to the queue, so it can be combined with individual files and with further `-dir` options.

3.5 Minimum system requirements

3.5.1 Operating system

Documented minimum: macOS 10.15 (Catalina) or later.

Note: the APIs the code uses have been available since macOS 10.10 (Yosemite) in theory (pthread_set_qos_class_self_np, C11 stdatomic.h), but no testing has been done below 10.15, so 10.15 is the supported minimum.

In practice: any macOS in use today (Big Sur, Monterey, Ventura, Sonoma) runs it without trouble.

3.5.2 Processor

Minimum: any 64-bit Intel or Apple Silicon (M1/M2/M3/M4) processor.

The program does almost nothing on the CPU while running: on the realtime thread it performs literally one memcpy. File decoding happens **before** playback starts, so CPU load during playback is negligible.

The official release is a universal binary, so it runs on both Apple Silicon and Intel machines. 32-bit processors are not supported.

3.5.3 Memory (RAM)

Because the program loads a whole playback segment (the tracks sharing one format) into memory and plays from there, the amount of RAM determines how large an album can be played back gapless.

- **Minimum (ordinary listening):** 4 GB. That is ample for a typical CD-quality (16/44.1 kHz FLAC) album — a 60-minute album takes roughly 400–500 MB decoded.
- **Recommended (hi-res / DSD):** 8–16 GB. A one-hour 24/192 kHz FLAC album takes 3–4 GB decoded. The memory image of DSD files starts at 30–40 MB per minute (DSD64) and is a multiple of that at DSD256.

Important: the memory peak is not the whole playlist but the **largest format segment** — the program loads segment by segment and frees the tracks at the end of each.

The mlock call (which stops the system from paging the data out to disk) can fail when little memory is free. In that case the program prints (mlock partial) instead of mlock active — meaning paging may occur during playback, which removes exactly the guarantee the whole-RAM model exists for. When that happens, it is worth splitting the playlist in two.

3.5.4 Audio hardware (DAC)

Minimum: the Mac's built-in audio device. **Recommended:** any external USB/Thunderbolt DAC compatible with the CoreAudio subsystem.

bpplay requests exclusive access (**hog mode**) to the DAC, disables the system mixer and — where possible — tries to establish an integer path. For one limitation of the macOS USB audio subsystem, see section 6.2.

Channel count: version 0.9 matches the channel count of the source file exactly to that of the output device. If the two differ (e.g. a 6-channel WAV on a stereo DAC), bpplay reports an error and does not play the file. There is no channel selection or downmixing — a deliberate decision, since downmixing would be a modification of the signal.

bpplay itself accepts source files up to **64 channels**; above that the header is rejected. That is only a theoretical ceiling, though: the real limit is always the DAC, because the player only accepts a format whose channel count matches the file exactly. bpplay does not restrict playback to even channel counts — in practice the number is even because audio devices generally offer even channel counts, so an odd-channel file usually has nothing to match against.

DSF (DSD) is stereo only, where the channel count must be 2.

A note on the click-free ramp at high channel counts:

the ramp described in section 5.2 works while one frame is at most 64 bytes. That means 16 channels with a 32-bit virtual format, 21 with 24-bit, and 32 with 16-bit. Above that playback works exactly as before, but the ramp switches off, and stopping and pausing become an immediate mute. On a macOS USB DAC the virtual format is typically float32, so the practical limit is 16 channels.

3.5.5 Storage

The program binary is a few hundred kilobytes. The music files of course need space; bpplay also handles mounted external and network drives.

4. Command line options

Option	Meaning
-l	List the available output devices, then exit.
-d <index>	Select the output device (DAC) by index.
-dir <folder>	Play all playable files of a folder, recursively, sorted by name.
-hu	Hungarian messages (English is the default).
-dop	Accepted for backward compatibility; has no effect. .dsf files switch to DoP on their own (see 6.1).
-v, --version	Print version and licence information, then exit.

4.1 Exit codes

Code	Meaning
0	The playback queue finished.
1	The user quit (q or Ctrl+C), or an error occurred (no such device, missing file argument, hog mode failure).

5. Transport controls during playback

The following keys work in the terminal during playback:

Key	Action
n	Next track (within the segment).
b	Previous track (within the segment).
space	Pause / resume.
q	Quit (with a clean, click-free stop).

The n and b keys act **within the format segment**: on the last track of a segment, n does not carry over into the next segment.

5.1 The status line

During playback bpsplay shows a status line: the index of the current track, the elapsed and remaining time within the track, and the elapsed and remaining time of the whole queue. While paused, the ► indicator at the start of the line changes to .

5.2 Click-free stopping and pausing

Stopping, the end of a track or of the queue, and pausing all use a **linear ramp** (1024 frames, about 23 ms at 44.1 kHz).

The ramp **never touches a music sample**: it repeats a copy of the last real frame written and scales that copy, so the audible content stays bit-exact and only material bpsplay itself appended after the content is ever scaled. The ramp state lives in the player, so a ramp that does not fit into a single CoreAudio block simply continues in the next callback.

For DSD/DoP the ramp is deliberately disabled. The top byte of a DoP word carries the 0x05/0xFA marker; scaling it would destroy both the marker and the DSD bits, and the DAC would drop out of DSD mode. With DSD, stopping is therefore an immediate mute.

6. Supported formats

Format	Details
WAV	16, 24 and 32-bit integer, plus 32-bit floating point. Float masters (e.g. Pyramix/Anubis) travel a direct route. Note: on a macOS USB DAC a 32-bit integer source is not fully bit-perfect on the output path because of the system's float intermediate (see 6.2); 16/24-bit content and 32-bit float pass untouched.
FLAC	16/24-bit, with Vorbis metadata and MQA detection.
M4A	ALAC (M4A), 16/24-bit, in the M4A container. Same bit-perfect, whole-track-in-RAM path as FLAC
AIFF / AIFC	16/24/32-bit, uncompressed only (NONE / sowt).
DSF (DSD)	DSD64 – DSD256 (the architecture scales to DSD512), packed as DoP. Stereo only. .dsf files play as DSD on their own.

6.1 DSD and DoP

macOS has no native DSD path, so **bpplay** uses the **DoP** (DSD over PCM) method: it packs the DSD bits into 24-bit PCM frames with an alternating marker byte, and the DAC recognises and unpacks it. The signal stays untouched throughout — the DSD frequency shown on the DAC's display confirms it.

.dsf files switch to DoP mode **automatically**; no separate option is needed. Safety is preserved: a DoP stream is only ever produced from a .dsf file, so noise can never be sent to a non-DSD file by accident.

The DoP rates

- DSD64 → 176.4 kHz DoP carrier
- DSD128 → 352.8 kHz DoP carrier
- DSD256 → 705.6 kHz DoP carrier

The higher DoP rates (352.8 and 705.6 kHz) do not pass on every DAC or USB chain. If the DAC cannot be set to the rate, bpplay reports it in plain words — DSD64 is the safest choice then.

bpplay trims the padding of the final DSF block using the sample count from the header, so no data that is not genuine content in DSD terms reaches the chain at the end of a track.

6.2 Known limitation — macOS USB and 32-bit integer

The macOS audio subsystem (CoreAudio) hands applications a floating-point (float32) intermediate format on USB-connected DACs, even when the DAC also offers an integer format. This is decided at system level, not in the player.

bpplay always reports this honestly in its output: the Virtual format: float line shows when the output virtual format is floating point. In practice:

- **16 and 24-bit content** (WAV, FLAC, AIFF, DSD/DoP, MQA): bit-perfect, because it fits exactly within the 24 bits of meaningful precision of the float32 intermediate. The MQA signalling passes intact and DSD appears as DSD on the DAC's display.
- **32-bit integer sources:** the lowest few bits may be lost in the float intermediate. This is a single, deterministic, controlled conversion — the Signal path: line then reports one controlled conversion rather than full bit-perfection. It affects few users, because 32-bit integer sources are rare (most studio masters are 24-bit integer or 32-bit float).

The limitation belongs to the macOS **USB** audio subsystem. It does not appear on a non-USB path — for example on a Ravenna/AES67 network audio interface — where the full signal passes untouched (see 11.7).

7. The whole-track-in-RAM model

bpplay loads the whole track (or segment) into memory at load time and pins it there (mlock). So there is no disk I/O during playback: the audio data reaches the DAC from RAM, with a single memory copy (memcpy).

7.1 Why is this good?

- There is no disk access during playback, so there are no I/O-induced CPU spikes and no electrical noise from them.
- mlock guarantees that the operating system cannot page the memory out — there is no unexpected latency and no dropout.
- A practical benefit: if the source disk (SSD) and the DAC share a bus or hub, there is still no disturbance during playback, because the disk **is not being read**.

7.2 About memory use

Memory use can be larger than the file size — this is normal and intentional. There are two reasons: the whole track is in RAM, and format matching also inflates the data to the output format. With DSD this is particularly striking:

- **DoP packing: $\times 1.5$** — one marker byte is added for every two DSD bytes ($2 \rightarrow 3$ bytes).
- **float32 path: $\times 1.333$** — if the HAL gives no integer path, samples go from 24 to 32 bits. This is lossless, because the mantissa of float32 is 24 bits.

The product of the two is exactly **2.0 $\times$** the raw DSD data. Example: a ~3.9 GB DSD256 file occupies roughly 7.5 GB in memory.

Important: this is neither a conversion nor a loss — the DSD bits travel untouched and the sound stays bit-perfect.

8. Mixed-format playlists

If the playlist (or folder) contains files of several formats, **bpplay** splits them into **format segments**. A segment = consecutive files of the same format (same sample rate, bit depth, channel count, integer/float type). Playback is gapless within a segment; at a segment boundary the DAC re-locks to the new sample rate — which means a short, unavoidable, but clean (click-free) pause.

bpplay shows which segment you are in with the [x/y] indicator. The n and b keys act within the segment.

Segmentation is decided by a lightweight read of the file headers, before loading — without decoding.

9. Drag-and-drop launching

The disk image includes a ready-to-use "bpplay drop" app — no Automator setup required. Drop a music file, several files, or an album folder onto its icon: a Terminal window opens, bit-perfect playback starts, and the transport keys (n/b/space/q) are available there too. It plays on the system's default output device (changeable in macOS System Settings → Sound).

Selecting a specific DAC (instead of the system default)

- "bpplay drop" always plays on the system default output. If you want to pin a specific device index (-d N), you have two options:
- From Terminal, by hand: `./bpplay -d N file.flac` (run `./bpplay -l` to see the indices)
- Or build your own Automator app with a fixed device index — `tools/bpplay-drop.sh` is the starting point, same as before

The ready-made launcher script is in the repository: [tools/bpplay-drop.sh](#)

9.1 Building the Automator application

- Open Automator and choose: New Document → Application.
- Find the „Run Shell Script” action and drag it onto the workflow.
- Set „Pass input” to „as arguments”.
- Paste in the body of `tools/bpplay-drop.sh` and set the two variables at the top of the file: BPPLAY (the full path to your bpplay binary) and DEVICE (the output device, e.g. -d 1; `./bpplay -l` gives you the index — leave it empty to play to the default output).
- Save it as an application (e.g. „bpplay drop”) on the Desktop.

From then on, dragging files or a folder onto the icon starts playback:

- a folder is handled in -dir mode automatically,
- .dsf files play as DSD automatically.

The script can also be run on its own from Terminal:

```
chmod +x tools/bpplay-drop.sh
./tools/bpplay-drop.sh /Users/you/Music/Album
```

9.2 Playback by right-clicking (Quick Action)

If the `~/Library/Services/` folder doesn't exist at all (normal on a user account that has never had a Quick Action/Service before):

- open Terminal and run `mkdir -p ~/Library/Services`, then you can copy "bpplay play.workflow" in there.

If you copied it in but "bpplay play" doesn't show up in the right-click → Quick Actions menu (or in the "Customize..." list): the background process serving the Services menu doesn't always notice a new item right away. Force it from Terminal:

/System/Library/CoreServices/pbs -flush — then open the right-click menu again.

The disk image also includes a "bpplay play.workflow" Quick Action, which appears in Finder's right-click menu. One-time install: copy it into ~/Library/Services/ (in Finder: Cmd+Shift+G, then ~/Library/Services). Afterwards, select one or more music files, or an album folder, anywhere, right-click, and choose "bpplay play" (possibly under the "Quick Actions" submenu). Starts in a Terminal window, same as drag-and-drop.

10. Troubleshooting

Symptom	Remedy
Hog mode cannot be acquired	Another application holds the DAC exclusively. Close the other players or applications using the audio output.
DSD does not start / noise	Check that the file really is a .dsf and that the DAC is DoP-capable. Enable native/Direct DSD mode on the DAC and use analogue volume (not digital).
High DoP rate error message	The DAC/USB chain cannot carry the DoP rate (e.g. DSD256 → 705.6 kHz). Try material at a lower rate (DSD64).
High memory use	This is normal, see section 7.2. Large DSD albums need more RAM.
(mlock partial) message	There is little free memory in the system. Close other applications, or split the playlist in two.
The folder plays in the wrong order	bpplay sorts by name. For album order, use leading zeros in the numbering (01_, 02_ ...).
„Channel count mismatch” error	The channel count of the source file does not match that of the output device. Version 0.9 does not perform channel selection; use a source matching the DAC's channel count.
Corrupt or unreadable file	bpplay skips such a file and reports it („Skipped (unreadable): ...”), then playback continues with the rest. A malformed WAV/AIFF/DSF header is rejected safely — one corrupt file does not stop playback.
32-bit music sounds quieter or less clean	On a macOS USB DAC a 32-bit integer source passes through the system's float intermediate (see 6.2). 16/24-bit material and 32-bit float are unaffected. If your source is 32-bit integer, convert it to 24-bit integer or 32-bit float — both travel bit-perfect.
The playlist is longer than 256 files	A single invocation plays at most 256 files; the program skips the rest and says so. Split it across several invocations.

11. Feature summary and validated state

11.1 Supported formats

- WAV (integer PCM 16/24/32-bit, plus 32-bit float)
- FLAC (lossless, with an embedded single-header decoder), 16/24-bit
- ALAC (lossless, in the M4A container, with Apple's own open-source decoder), 16/24-bit
- AIFF / AIFC (uncompressed PCM, big-endian, converted once at load time)
- DSF (DSD), played as DoP — DSD64, DSD128 and DSD256, stereo
- Supports DAC-side MQA decoding (bplist detects, it does not decode)

11.2 The signal path

- Exclusive hardware access (hog mode) — no other application can reach the DAC
- Direct CoreAudio HAL access through a realtime IOProc callback that performs nothing but memcpy
- No software volume, no dither, no resampling, no DSP of any kind
- Automatic sample-rate switching to match the file, so the system's resampler never runs
- A single, lossless format-matching step at load time

11.3 The whole-track-in-RAM model

- The entire format segment is loaded into memory before playback starts
- The buffer is pinned with mlock, so the operating system cannot page it out
- Zero disk I/O and zero decoding during playback

11.4 DSD handling

- Automatic DoP: the .dsf extension enables DoP packing without any option
- DSD64 / DSD128 / DSD256, rate-independently (the DoP carrier scales with the rate)
- A clear error message if the DAC cannot be set to the DoP rate

11.5 Playback and convenience

- Gapless playback between tracks of the same format
- Recursive folder traversal in album order, to unlimited depth
- Automatic segmentation of mixed-format playlists
- Click-free ramping on stop and pause, without touching a music sample
- Sleep prevention during playback (the Mac will not fall asleep mid-album)
- Key control from Terminal

- Bilingual (English/Hungarian) messages

11.6 Engineering character

- One C file and three headers, around two thousand lines, plus a single embedded FLAC decoder — zero external build dependencies
- Builds with a single make, cleanly, without warnings
- Every line of the signal path is visible and auditable

11.7 Validated state

bpplay is **validated up to DSD256**, through Gustard XMOS and Amanero Combo384/768 interfaces (with TT and Gustard DACs). The chain was also tested in three connection topologies: the DAC plugged directly into the Mac's USB port, through a Sonnet Tech Thunderbolt 5 hub, and through a no-name USB hub; with both expensive and cheap cabling. Bit-perfect behaviour was identical in all three cases: the bit path is independent of the connection method.

Release 0.9 was also tested in a fourth, **non-USB topology**: over Ravenna/AES67 network Merging interfaces — with Merging Anubis and Merging NADAC DACs — bpplay worked flawlessly. This is independent confirmation that the float-intermediate limitation described in 6.2 is specific to the macOS USB audio subsystem: it disappears on another output path.

Using Ravenna/AES67 requires a working Core Audio setup: the Merging virtual audio device (with the manufacturer's Ravenna/AES67 driver) must be correctly configured in the system and must appear as an available output device. bpplay picks it up in the -l list and from then on treats it like any other CoreAudio device — it performs no network configuration itself.

Building release 0.9 as a universal binary (Apple Silicon + Intel), as well as playback, pause/resume and format detection on a real DAC (Gustard), were all verified before release.

12. About bpplay — philosophy and architecture

Condensed summary. The full bilingual version is in the repository: docs/[about-bpplay.md](#)

Blog series, part 1:

<https://medium.com/@mediaengineering/bpplay-uncompromising-computer-audio-architecture-and-software-minimalism-fbcc30753b28>

Blog series, part 2:

<https://medium.com/@mediaengineering/bpplay-what-happens-under-the-hood-85d5159bb739>

Blog series, part 3:

<https://medium.com/@mediaengineering/bit-perfect-memory-based-audio-file-playback-in-2026-7199a2921b53>

Blog series, part 4:

<https://medium.com/@mediaengineering/linux-and-windows-environment-optimization-possibilities-for-high-end-audio-playback-c2e6c8c1fee0>

Blog series, part 5:

<https://medium.com/@mediaengineering/bit-perfect-playback-what-does-it-really-mean-and-when-does-it-actually-matter-9b98efaca480>

12.1 The starting point: analogue minimalism

The guiding principle behind My Reel Club® recordings is simple: the best thing you can do to a recording is often nothing at all. Few microphones, a direct stereo arrangement, no post-production — so it is known exactly what is in the groove or on the tape. **bpplay** translates that principle into the world of computer-based digital playback.

bpplay does not promise that it „sounds better“. It promises that **what happens during playback is knowable**. Every step of the chain is visible and verifiable: one C file and three headers, around two thousand lines in total, plus a single embedded FLAC decoder — no external build dependencies, and short enough to be read end to end.

12.2 What a player usually does — and what bplay does not

Listening through an ordinary macOS application, the sound does not reach the DAC directly but passes through the system audio mixer. There, resampling, floating-point mixing and software volume control may take place. In most situations these are convenient and useful steps — but each of them is a place where the signal can change, and where the processing can generate incidental noise.

bplay bypasses that entire intermediate layer. It takes exclusive control of the DAC (hog mode), disables mixing, and hands the samples over through a direct realtime callback (IOProc). Volume and every further step are left to the devices designed exactly for it: the DAC and the amplifier.

12.3 The bit-perfect signal path

The bplay signal path consists of four steps that run only once, all of them completed before the DAC is started:

5. **Device acquisition (hog mode).** The program becomes the exclusive owner of the DAC; the system mixer is disabled.
6. **Format negotiation with the HAL.** bplay asks the hardware: is there an integer physical format the source material can be fitted into? If so, it sets it; if the device can only do float32 at that rate, it accepts that too — and states which route it is taking.
7. **Loading and a single format-matching step.** The whole segment is loaded into memory; if needed, one lossless conversion takes place — still before playback, on a non-realtime thread.
8. **mlock.** The audio buffer stays physically in RAM.

During playback the IOProc performs exactly one operation: memcpy. There is no floating-point arithmetic, no allocation, no file reading and no lock. Loading and playback are two entirely separate phases: what is permitted at load time (decoding, DoP packing, format matching) is forbidden on the realtime thread.

12.4 The electrical benefit of the RAM model

A conventional player reads continuously from storage, decodes and manages buffers; every such operation puts a short, spike-like load on the machine's power supply. The read spike of a modern NVMe SSD can jump from 15 mA to 2 A within microseconds — a rate of change close to two million amperes per second (di/dt) — which can generate high-frequency RF/EMI noise. The whole-RAM model removes that noise source at its origin.

bplay does not claim that this „makes it sound better“. The whole-RAM model removes the noise source itself; whether that has an audible consequence depends on the design of the DAC.

12.5 DoP and DSD

DSD stores the signal not in time-domain PCM samples but in 1-bit samples at an extremely high frequency (11.2896 MHz per channel for DSD256). That bit density cannot be carried directly over USB audio interfaces, so `bpplay` uses the **DoP** (DSD over PCM, v1.1) protocol: it packs 16 DSD bits into a 24-bit PCM sample and puts a marker byte alternating between 0x05 and 0xFA into the top 8 bits. The carrier PCM rate is one sixteenth of the original DSD rate.

The alternating marker is a self-checking handshake: if the DAC supports DSD over DoP, it unpacks the bits; if not, it stays passive and treats the data as ordinary PCM. The 16 DSD bits are bit-for-bit the same as in the DSF file — the DoP frame is only the transport protocol, not the content.

12.6 Format handling and gapless playback

Gapless playback is a given: the `IOProc` copies the data of the next track as soon as the previous one runs out — and does so **within the same callback**, so a track boundary never coincides with a callback boundary. This can only be maintained between files of the same format, however, because changing the sample rate requires a hardware reconfiguration of the DAC.

`bpplay` therefore splits the playlist into format segments before loading, using a lightweight header read. `bpplay` does not hide what cannot be avoided — it tells you why the pause is there and where.

The big-endian byte order of **AIFF** is handled at load time by a per-sample byte swap; this is arithmetically lossless. FLAC files containing **MQA** are passed on as ordinary FLAC; the actual MQA decoding — if the DAC supports it — happens in the DAC.

12.7 What is deliberately left out

- **ALAC** — lossless just like FLAC, so the decoded PCM is bit-identical: there is no sound quality advantage. Its container (MP4/M4A) is also orders of magnitude more complex, and the „easy route“ (native CoreAudio decoding) would lead exactly through the opaque layer `bpplay` bypasses. Anyone with an ALAC library can convert it losslessly to FLAC.
- **.dff / .wsd** — possible in principle (the DoP back end is format-independent), but of lower priority given their marginal practical occurrence; DST-compressed `.dff` would require a separate decoder.

The missing functions (library management, graphical interface, network streaming, signal processing, volume) are not omissions: each of them would be one more place where the signal could change or noise could arise between the file and the DAC.

12.8 The computer as a noise source — and the clock question

Most demanding USB DACs today use **asynchronous USB**: there **the DAC is the clock master**, its own crystal oscillator times the conversion, and its input buffer isolates the timing

inaccuracies of the computer. `bpplay` therefore does not provide — and does not claim to provide — a better clock.

What it can genuinely help with is the **electrical coupling path**: fewer di/dt transients on the computer side mean less EMI and power-rail noise, which could otherwise degrade the phase noise of the DAC's own clock and its noise floor — provided a coupling path exists (shared ground, bus power, incomplete isolation). A galvanically well-isolated, self-powered DAC breaks that coupling anyway.

12.9 A reference, not a competitor

bpplay gives up the attractive interface and the convenience features, and in exchange offers a **reference**: a fixed point to measure against. Because the route from the file to the DAC is known, deterministic and free of intervention, when getting to know a new DAC the result you hear is far more cleanly the work of the DAC — the software does not interfere with the judgement.

bpplay was created with the assistance of a large language model; the architecture (whole-RAM model, hog mode, intervention-free signal path) and the features followed from hi-fi principles and practical experience, not from the model. The model was a partner in the implementation — in the CoreAudio API, in the bit-exact code, in the debugging.

13. Licence terms and copyright notice

Software name: bpplay (Core edition), 0.9

Copyright © 2026 Koscsó Ferenc / Koscsó Media Engineering Kft.

This software is free software; it may be redistributed and/or modified under the terms of the GNU General Public License as published by the Free Software Foundation, **version 3 or (at your option) any later version**.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the full text of the GPLv3 licence for details.

User rights: you are free to run, study, modify and copy the software. When distributing the software or works derived (modified) from it, however, the source code must be made public, and the derived work must likewise be published under the GPLv3 licence.

The full, official text of the licence: <https://www.gnu.org/licenses/gpl-3.0.html> — in the repository: LICENSE

Third-party component:

dr_flac (David Reid – dr_libs, 0.13.4) — the component responsible for decoding FLAC files.

Published under a choice of **public domain (Unlicense)** or **MIT-0**; the full licence statements are at the end of the src/dr_flac.h file. Both options are GPL-3.0 compatible. Details:

THIRD_PARTY_NOTICES.md

Disclaimer:

The software — in particular hog mode (exclusive device access) and memory pinning (mlock) — communicates directly with the operating system and the hardware. Although the code was developed with the greatest care and has been through testing, use of the software is at your own risk. The author cannot be held liable for any data loss, hardware damage or possible damage to hearing (which may result from a sudden high volume level).